The ensemble adjoint of JAX-DSMC as implemented

2026-09-17; shape optimiser and status updated 2026-09-24

This note states, equation by equation, what the particle-carried ensemble adjoint in `jax_dsmc/ensemble_backw` (host reference) and `jax_dsmc/ensemble_device.py` (JIT port) computes, where each term comes from in the reference paper ("P3", `papers_reference/arXiv-2603.20946v2/main.tex`, section 5, *Ensemble-Adjoint DSMC Method With Inflow Boundary Conditions*), and where in the code it lives. Equation labels of the form P3 eq. `label` are the `\label{eq:label}` names in P3's source. Numbers quoted in section 6 are copied from `docs/ADJOINT_BRANCH_REVIEW.md` ("review") sections 5.7, 5.12 and 6. Everything marked **planned** is not in the code.

Reading order. Readers who know CFD but not DSMC should start with appendix A, which writes out the forward solver (NTC collisions, the VSS model, sources, particle tracking through cut cells) as the code does it. Section 1 is the compact notation the adjoint uses. The shape optimiser and the status of the shape work are in sections 7.1 and 7.2.

1 Forward step and notation

One forward step $k \to k+1$ of `simulate._step` (`jax_dsmc/simulate.py`) is, in order,

$$(x_k, v_k, a_k) \xrightarrow{\text{move + walls}} (x_k^m, v_k^m, a_k^m) \xrightarrow{\text{inject}} (x_k^m, v_k^m, a_k^m) \cup \{\text{newborns}\} \xrightarrow{\text{sort}} \text{cell tables} \xrightarrow{\text{collide}} (x_{k+1}, v_{k+1}, a \tag{1.1}$$

with $x_{k+1} = x_k^m$ (collisions never move particles) and a the boolean alive mask over a fixed number N of particle slots. This is the reverse of P3's order (Algorithm 1 there: collide, then P3 eq. `x_update` transport), which flips the backward order in section 3. The superscript m denotes the *post-move, post-injection, pre-collision* state: it is what `EventRecord.x`, `.v`, `.alive` store (`simulate.EventRecord`), while `EventRecord.v_post` $= v_{k+1}$.

Move and walls. Free flight $x^m = x + \Delta t\, v$ with chronological first-event tracking on every face (`wall_adjoint._advance_impl`); specular and periodic faces are deterministic maps; an absorbing/outflow face sets $a_i^m = 0$; a diffuse face re-emits with the accommodation-mixed kernel of section 4. The flight budget is the ε-regularised time $\tau_i = \max(\Delta t + \varepsilon z_i, 0)$, $z_i \sim \mathcal{N}(0,1)$ (P3 section 4's random-time wall, P3 eq. `x_update_new`); $\varepsilon = 0$ recovers the deterministic step.

Inject. For every `INFLOW` face f the expected count is (`inflow.target_count`)

$$\bar{N}_f(\theta) = n_{\rho,f}\, v_{s,f}\, F(s_f)\, A_f\, \Delta t / F_{\text{num}}, \qquad v_{s,f} = \sqrt{2 k_B T_f / m}, \quad s_f = \frac{u_f \cdot n_f}{v_{s,f}}, \tag{1.2}$$

where F is the untruncated flux integral (`inflow.flux_integral`) and n_f the inward normal. The realised count is the stochastic rounding $N_f = \lfloor \bar{N}_f \rfloor + \mathbf{1}[u < \text{frac}\, \bar{N}_f]$ (`sampling.stochastic_round`). Each newborn draws a velocity v_b from the normalised inward flux density b_f of (4.4), a face position, and a first-move fraction $\xi \sim \mathcal{U}[0, \Delta t]$, and is placed at $x_f + \xi v_b$ with the same wall tracking as everybody else (`inflow.inject`, `track=True`). This is P3 eq. `ensemble_inflow_left_state`

with P3's $g_{k,i}$ replaced by the drifting half-Maxwellian sample. The record keeps v_b *as sampled* (`born_v`, the score's argument), the face (`born_face`) and the newborn's own birth-step wall hits (`born_walls`).

Collide (NTC, VSS). Per cell c with N_c members and volume V_c, the attempt count is the stochastic rounding of

$$\bar{n}_{\mathrm{att},c} = \tfrac{1}{2} N_c (N_c - 1)\, \Sigma_c\, \Delta t\, F_{\mathrm{num}} / V_c \qquad (\texttt{ntc.attempt_count}), \tag{1.3}$$

where Σ_c is the per-cell majorant (thermal a-priori or certified, `ntc.thermal_majorant`, `ntc.certified_majora` recorded as `EventRecord.sigma`. Each attempt a of cell c (`ntc._slot`, one scan slot for all cells) draws an unordered pair (i, j) uniformly, accepts it with probability

$$p_{ij} = \min\big(q(g_{ij})/\Sigma_c,\, 1\big), \qquad q(g) = \sigma_T(g)\, g = C_{\mathrm{VSS}}\, (g^2)^{1-\omega}, \quad g = |v_i - v_j| \qquad (\texttt{vss.collision_rate}), \tag{1.4}$$

and on acceptance replaces (v_i, v_j) by the VSS post-collision pair $(v_i', v_j') = \mathrm{scatter}(v_i, v_j; u_\chi, u_\epsilon)$ (`vss.scatter`, deflection law in α). Attempts are processed in slot-index order, so a particle may collide more than once per step and the a-th attempt sees the velocities left by attempts $< a$. The record stores, per slot and cell, $(i, j, \mathrm{accept}, v_i^{\mathrm{pre}}, v_j^{\mathrm{pre}}, v_i^{\mathrm{post}}, v_j^{\mathrm{post}})$ (`ntc.PairRecord`). The accepted pairs are exact samples of the collision measure $q(g) f f$ if and only if the majorant is never violated (`Diagnostics.majorant_violations` $= 0$).

RNG contract. Every stage of every step draws from

$$\texttt{key_for}(\kappa, k, s) = \mathrm{fold_in}\big(\mathrm{fold_in}(\kappa, k), s\big), \qquad s \in \{\texttt{MOVE}, \texttt{WALL}, \texttt{NTC_ROUND}, \texttt{PAIR}, \texttt{ACCEPT}, \texttt{SCATTER}, \texttt{INFLOW}\}, \tag{1.5}$$

(`jax_dsmc/rng.py`), with the attempt-slot index folded in once more for per-slot streams. The capture (`simulate.run(capture=True)`) does not change any stream or any arithmetic of the forward (`tests/test_ensemble_backward.py::test_capture_is_consistent_with_the_states`). The backward's own randomness (virtual partners, section 3.3.3) uses a separate key.

2 Objectives

Terminal objective (implemented).

$$J = \frac{1}{\mathcal{N}} \sum_{i:\, a_{M,i}=1} r(x_{M,i}, v_{M,i}), \tag{2.1}$$

which is P3 eq. `ensemble_inflow_objective` with one difference: P3 fixes $\mathcal{N} = N_0 + M(N_L + N_R)$; here $\mathcal{N}$ is the `normalization` argument of `ensemble_backward.backward` and is a constant of the run (the nominal particle number n_{nominal} on the inlet shock, N on Couette and the relaxation). The per-particle reward r is the callable `phi(x, v)` on the host; the device takes the already-divided vector `phi_final` $= r/\mathcal{N}$.

Value form. The device and the host both accept an arbitrary per-particle terminal vector, which is used for non-additive objectives by the delta method: for the density ratio $J = N_{\mathrm{down}}/N_{\mathrm{up}}$ (`examples/ensemble_inlet_shock.py::objective_terminal`)

$$\lambda_{M,i} = \frac{\mathbf{1}[x_{M,i} \in \mathrm{down}]}{N_{\mathrm{up}}} - \frac{N_{\mathrm{down}}}{N_{\mathrm{up}}^2}\, \mathbf{1}[x_{M,i} \in \mathrm{up}], \tag{2.2}$$

with the two totals frozen at their realised values ($O(1/N)$ bias, review section 6).

Windowed and per-hit rewards (value form, implemented 2026-09-18). For $J = \mathcal{N}^{-1}\sum_k \sum_i \phi_k(x_{k,i}^{\text{post}}, v_{k,i}^{\text{post}}) + \mathcal{N}^{-1}\sum_k \sum_{\text{hits } j \text{ of step } k} r_{k,j}$ the ensemble adjoint acquires a running source, in the sense of P3 section 7.5 (P3 eq. `running_alpha`–P3 eq. `running_beta` are its pathwise analogue). `backward` and `backward_virtual` take `phi_step` $= \phi_k/\mathcal{N}$ (`f64[M, N]`, at each step's post-collision state – the state `simulate.run`'s `reward` hook sees) and `hit_reward` $= r_{k,j}/\mathcal{N}$ (`f64[M, K, N]`, one slot per tracker pass; for the lane tracker the per-step totals `shape.HitRewards.totals()`, `f64[M, 1, N]`, which is all the move pull-back sums). Before the pull-back of step k

$$\mu_{k,i} \leftarrow \mu_{k,i} + \phi_k(x_{k,i}^{\text{post}}, v_{k,i}^{\text{post}})/\mathcal{N}, \tag{2.3}$$

and on the move pull-back of step k (survivors only; a newborn's rewards go to its source score)

$$\mu_{k-1,i} = \lambda_{k,i}^m + \frac{1}{\mathcal{N}}\sum_j r_{k,j,i}. \tag{2.4}$$

The weight of the wall-kernel score (4.3) of hit j is the reward-to-go *from that draw onward*,

$$w_{k,j,i} = \lambda_{k,i}^m + \frac{1}{\mathcal{N}}\sum_{j' \geq j} r_{k,j',i}, \tag{2.5}$$

the hit's own reward included – it is a function of the drawn velocity (leaving it out made the Couette windowed-shear gradient with respect to the wall speed read -0.0004 against an FD of -0.0165; with it, -0.0177 ± 0.0014). The terminal vector `phi_final` is unchanged and equals $\phi_M/\mathcal{N}$ for a windowed objective whose last step is the final one. Gates: G1 (windowed v_y^2, `speed`, $z = -0.05$) and G2 (windowed shear as a per-hit reward, `speed` $z = -0.77$, T_{left} $z = +0.46$), 48 seeds each against CRN finite differences (`examples.ensemble_couette --objective`).

3 The ensemble adjoint as implemented

3.1 Carried values and the terminal condition

P3 defines $\lambda_k(x, v) = \delta J/\delta f_k(x, v)$ with the terminal condition P3 eq. `ensemble_inflow_terminal_adjoint`, $\lambda_M = r/\mathcal{N}$ in Ω and 0 outside. The code represents λ_k by *one scalar per particle slot at that slot's own state* (route (b) of review section 5.1; P3 section 5.4 uses retained subsets and reconstructs the field at every retained state instead):

$$\mu_i := \lambda_M(x_{M,i}, v_{M,i}) = a_{M,i}\, r(x_{M,i}, v_{M,i})/\mathcal{N} \qquad (\texttt{backward: mu = where(alive_final, phi/normalization}$$ (3.1)

Going backward through step k, the sweep turns μ (values at t_{k+1}, i.e. at (x_k^m, v_{k+1})) into $\lambda_i^m := \lambda(x_{k,i}^m, v_{k,i}^m)$ at the post-move state, records what sources and walls need there, and then into μ at t_k. The device port emits exactly λ^m per step as `Backward.lam[M, N]`.

3.2 Move pull-back

P3's transport pull-back P3 eq. `ensemble_inflow_transport_adjoint` is $\tilde{\lambda}_{k+1}(x, v) = \mathbf{1}[x + \Delta t\, v \in \Omega]\, \lambda_{k+1}(x + \Delta t\, v, v)$. Because the value is carried on the particle that made the move, this is the identity on survivors:

$$\mu_i^{(k)} = \lambda_k(x_{k,i}, v_{k,i}) = \begin{cases} \lambda_i^m & a_{k,i}^m = 1 \text{ and slot } i \text{ was not born in step } k, \\ 0 & \text{otherwise (absorbed during the move, or born this step),} \end{cases} \tag{3.2}$$

(`backward: mu = where(alive & ~born, lam, 0)`; device `_step_pullback`, `_step_pullback_virtual`). The identity holds because specular and periodic faces are deterministic maps (the value moves

with the particle) and a diffuse face is a Markov kernel whose realised re-emission is an unbiased sample of it, so $\mathbb{E}[\lambda(\text{after hit}) \mid \text{before hit}]$ is estimated by the one carried value (review section 5.6). No ε enters: the regularised flight time is parameter-free randomness to this estimator for every control in section 6.

3.3 Collision pull-back

P3 pulls the adjoint through the collision step with the adjoint of the linearised operator, P3 eq. `ensemble_collision_adjoint_inflow`,

$$(\mathcal{L}_f^*\psi)(v) = \int_{\mathbb{R}^3} f(w) \int_{S^2} q(\sigma)\big[\psi(v') + \psi(w') - \psi(v) - \psi(w)\big]\,\mathrm{d}\sigma\,\mathrm{d}w, \tag{3.3}$$

and the one-step recursion P3 eq. `ensemble_adjoint_recursion_inflow`, $\lambda_k = \tilde{\lambda}_{k+1} + \Delta t\,\mathcal{L}_{f_k}^*\tilde{\lambda}_{k+1}$. Because collisions here come last in the step, the code applies the collision pull-back to μ (the values at t_{k+1}) *before* the move pull-back of section 3.2; the object it produces is λ^m at the post-move state. In the code q is the VSS rate of (1.4) and the angular measure is the VSS deflection law, not P3's Maxwell kernel; with realised or NTC-weighted pairs the kernel never has to be integrated explicitly.

Three pull-backs are implemented and selected process-wide by `ensemble_backward.configure(pullback, S, K, nodes)`. Throughout, $\Lambda^{(\cdot)}$ denotes the per-cell field interpolant of section 3.4 built from the current carried values on the current node velocities.

3.3.1 Sequential (exact discrete adjoint of the NTC step; reference)

Walk the accepted attempts of each cell in *reverse* slot order. At attempt a with pair (i, j), pre-collision velocities $(v_i^{\mathrm{pre}}, v_j^{\mathrm{pre}})$ from the record, current carried values λ_i, λ_j (which at that point are the value function right after attempt a, at the post-collision states) and current node velocities $V^{(a)}$ (the population as it stood right after attempt a):

$$\begin{aligned} \lambda_i &\leftarrow \lambda_i + \lambda_j - \Lambda^{(a)}(x_j, v_j^{\mathrm{pre}}), \\ \lambda_j &\leftarrow \lambda_j + \lambda_i - \Lambda^{(a)}(x_i, v_i^{\mathrm{pre}}), \qquad \text{then } V_i^{(a-1)} \leftarrow v_i^{\mathrm{pre}},\ V_j^{(a-1)} \leftarrow v_j^{\mathrm{pre}}, \\ \Phi_a &:= \lambda_i + \lambda_j - \Lambda^{(a)}(x_i, v_i^{\mathrm{pre}}) - \Lambda^{(a)}(x_j, v_j^{\mathrm{pre}}), \end{aligned} \tag{3.4}$$

where the right-hand sides use the values before the update (`ensemble_backward._collision_pullback_sequer` device `ensemble_device._slot_events` inside a reverse `lax.scan` over slots). This is the four-term sum of (3.3) with the realised pair as the single Monte Carlo sample of the collision measure and the two "+" terms read from the carried values (no interpolation), so the only interpolated quantity per accepted attempt is the partner's pre-collision "ghost" (review section 5.1). With `GHOST_FIELD = "current"` the ghost is read from the field as chained so far, which makes the sweep the exact discrete adjoint of the sequential NTC step (`test_sequential_pullback_reproduces_the_pre_` Φ_a is the pair's *four-term* (`Pairs.four_term`, device `Backward.four_term`), weight 1 per realised pair. Its per-seed noise grows with the collisions per particle per step and does not fall with Δt (review section 5.10).

3.3.2 Δt-linearised (reference only, biased $O(\Delta t)$)

All accepted attempts at once, every ghost from the end-of-step field Λ_{k+1} fitted on (x^m, v_{k+1}) with values μ:

$$\lambda_i^m = \mu_i + \sum_{a \ni i} \Big[\Lambda_{k+1}(x_{j_a}, v_{j_a}^{\mathrm{post}}) - \Lambda_{k+1}(x_{j_a}, v_{j_a}^{\mathrm{pre}})\Big], \qquad \Lambda_{k+1}(x_j, v_j^{\mathrm{post}}) = \mu_j \text{ when } a \text{ was } j\text{'s last accepted attemp} \tag{3.5}$$

(`ensemble_backward._collision_pullback`, `configure("linearised")`). This is P3 eq. `ensemble_adjoint_r` quadratured on the realised pairs, and it is what P3's single-step recursion literally says; it is -8.6% biased on the relaxation arbiter at 0.16 collisions/particle/step (review section 5.12) because a third of the accepted pairs have a member colliding again in the same step and the end-of-step field is not the value after *their* attempt. Not shipped.

3.3.3 Virtual partners, substepped (shipped: `VIRTUAL = (S, K) = (4, 8)`, `VIRTUAL_NODES = "staged"`)

The backward collision operator $e^{\Delta t \mathcal{L}^*}$ is applied in S substeps of $\Delta t/S$ each, going backward $s = S-1, \dots, 0$. Substep s uses node velocities $V^{(s+1)}$ (below) and one smooth per-cell fit $\Lambda^{(s)}$ of the current λ on those nodes. For every alive particle i in cell c and each of K virtual partners j_κ drawn uniformly among the *other* alive members of c, with fresh VSS uniforms $(u_\chi, u_\epsilon)_\kappa$:

$$\lambda_i \leftarrow \lambda_i + \sum_{\kappa=1}^{K} w_{i\kappa} \Big[\Lambda^{(s)}(x_i, v_i') + \Lambda^{(s)}(x_{j_\kappa}, v_{j_\kappa}') - \Lambda^{(s)}(x_i, v_i) - \Lambda^{(s)}(x_{j_\kappa}, v_{j_\kappa}) \Big], \tag{3.6}$$

$$w_{i\kappa} = \frac{q(|v_i - v_{j_\kappa}|)}{\Sigma_c} \cdot \frac{2\, n_{\mathrm{att},c}}{N_c} \cdot \frac{1}{KS}, \qquad (v_i', v_{j_\kappa}') = \mathrm{scatter}(v_i, v_{j_\kappa}; u_\chi, u_\epsilon)_\kappa, \tag{3.7}$$

where $v_i = V_i^{(s+1)}$, $n_{\mathrm{att},c}$ is the realised attempt count of the cell in this step (`pairs.pi < N`, summed over slots) and N_c its alive occupancy (`ensemble_backward._collision_pullback_virtual`; device `ensemble_device._substep`, `_partner_pass`). All four values come from the same fit, so their interpolation errors cancel to leading order. This is P3's Monte Carlo collision correction P3 eq. `MC_collision_adjoint_inflow`, $\widehat{C}_{k,i} = \Delta t\, \rho_k^j Q_0\, N_{\mathrm{col}}^{-1} \sum_s [\cdots]$, with $N_{\mathrm{col}} = K$ and the Maxwell prefactor $\Delta t\, \rho Q_0$ replaced by the NTC expectation weight $(q/\Sigma_c)\,(2n_{\mathrm{att},c}/N_c)$: the expected number of collisions a particle suffers in the step is $2n_{\mathrm{att},c}/N_c$ attempts times the acceptance q/Σ_c, and the realised attempt count already carries Δt, F_{num}, V_c and Σ_c from (1.3). No realised pair enters except through $n_{\mathrm{att},c}$. The substep count S is not in P3; it converts the single-step recursion into an $O(\Delta t/S)$ splitting (review section 5.12: $+1.8\%$ at $S=2$, $+0.7\%$ at $S=4$, $+0.1\%$ at $S=8$ on the relaxation arbiter).

Staged nodes. The NTC step processes slots in index order, so the population at the boundary after a fraction b/S of a cell's attempts is v^{pre} overwritten, for each particle, by the post-collision velocity of its last accepted slot before that boundary (`ensemble_backward._staged_velocities`, device `ensemble_device.staged_velocities`; tested against a sequential replay). The nodes of substep s are $V^{(s+1)}$, the state at the end of its own slot range, with $V^{(0)} = v^m$ and $V^{(S)} = v_{k+1}$. After the update (3.6) a particle whose velocity differs across the boundary moves its value through a *refitted* field $\Lambda'^{(s)}$ (current λ, nodes $V^{(s+1)}$):

$$\lambda_i \leftarrow \lambda_i + \Lambda'^{(s)}(x_i, V_i^{(s)}) - \Lambda'^{(s)}(x_i, V_i^{(s+1)}) \qquad \text{if } V_i^{(s)} \neq V_i^{(s+1)} \tag{3.8}$$

(`_boundary_move` on the device). With `nodes = "post"` every $V^{(s)}$, $s \geq 1$, equals v_{k+1} and the field inside the step is frozen ($+5$–6% biased, review section 5.12); the runners pass `"staged"` (note that `backward_virtual`'s own default argument is `"post"`).

Kernel-parameter term. Each virtual pair also returns its four-term $\Phi_{i\kappa}$ (the bracket of (3.6)) with weight $\frac{1}{2} w_{i\kappa}$ (`Pairs.weight`; the $\frac{1}{2}$ because every unordered pair is generated from both members), and the device accumulates per step

$$\texttt{Backward.kernel}_k = \sum_{i,\kappa} \tfrac{1}{2}\, w_{i\kappa}\, \Phi_{i\kappa}\, \partial_\theta \log q(v_i, v_{j_\kappa}) \tag{3.9}$$

when a `pair_score` $= \partial_\theta \log q$ is supplied (used by `examples/ensemble_relaxation.py` for `diam` and `omega`).

Stability. The quadrature noise injected by (3.6) is smoothed by the next fit; with too few partners for the cell occupancy the carried field grows geometrically over a long horizon ($K = 2$ at 40 particles per cell blew up by 3×10^6 over 300 steps; $K \geq 8$ decays, review section 5.12). `examples.ensemble_inlet_shock.device_backward` refuses a backward whose rms λ grew more than `GROWTH_LIMIT` $= 100\times$ over the horizon.

3.4 The field interpolant

P3 uses a weighted local *affine* fit in (x, v_1, v_2, v_3) on retained particles (section 5.4 there). Here the interpolant is cell-local and quadratic, in two forms: since 2026-10-11 the collision pull-back's value field is **one polynomial per collision cell** (this section's first part); the slopes of the shape adjoint and the host backward keep the per-query fit of the second part, which was also the device pull-back's until then.

The per-cell fit (the device virtual pull-back). The virtual pull-back evaluates the field at $(3 + 2K)N$ queries per substep: each node at its own state, the two post-collision ghosts of each of its K virtual partners, and the two states of the boundary move. Each collision cell c holds n_c alive nodes at scaled states $\tilde{p}_n = (x_n/\Delta x,\ v_n/v_{\text{scale}})$ with carried values λ_n. With the node mean $\bar{p}_c = n_c^{-1} \sum_{n \in c} \tilde{p}_n$ and the offsets $o = \tilde{p} - \bar{p}_c$, the cell's coefficients solve the unweighted least-squares problem

$$\Big(\sum_{n \in c} \phi_n^{\mathcal{K}} \phi_n^{\mathcal{K}\top} + \rho\, \bar{d}\ \mathrm{diag}(0, 1, \ldots, 1) + I_{\bar{\mathcal{K}}} \Big) \beta_c = \sum_{n \in c} \phi_n^{\mathcal{K}} \lambda_n, \qquad \phi_n^{\mathcal{K}} = \phi(o_n)\big|_{\mathcal{K}_c}, \tag{3.10}$$

with the design ϕ of (3.14) (`cell_fit.features(.., "quad")`, the same 14 columns in 2-D) restricted to the kept columns $\mathcal{K}_c$, the dropped columns $\bar{\mathcal{K}}$ masked to an identity block (their coefficients are 0), and every query in c reads

$$\Lambda(x_q, v_q) = \phi(\tilde{p}_q - \bar{p}_c) \cdot \beta_c, \qquad \nabla\Lambda = \partial_o \phi(\tilde{p}_q - \bar{p}_c) \cdot \beta_c \,/\, \text{scale}. \tag{3.11}$$

The order guard. Group the columns by polynomial order $g = 0, 1, 2$ with cumulative counts $m_0 = 1$, $m_1 = 1 + D + 3$, $m_2 = 14$ (2-D); keep the orders $g \leq g_c$ with $g_c = \max\{g : 1.5\, m_g \leq n_c\}$ (`adapt1.5`): a cell of fewer than 21 nodes fits a plane, fewer than 9 its mean, an empty cell nothing (value 0; its matrix the identity).

The shipped basis and the leverage guard (2026-10-19). The design above is the quadratic `quad`; the shipped `cubicv` adds the cubic monomials of v (24 columns in 2-D, so `adapt1.5` needs 36 nodes for the full order) and `guard32` adds a per-query test: with M_g the normal matrix of order g, the order used at a query is the highest g (within the `adapt1.5` cap) whose leverage $h_g = \phi_g^\top M_g^{-1} \phi_g \leq 32$ — the variance of the fitted value in units of one node's noise — and the cell mean if none. Unguarded, the cubic extrapolates at post-collision queries in the empty velocity gap between a M 10 beam and the wall-reflected gas ($h \sim 10^3$) next to bodies under ~ 2 cells thick, and the backward recursion compounds it (squash M 10 $\mathrm{d}D/\mathrm{d}s = -19.6$ against FD $+1.48$; `optm` nose spikes). Guarded, the thin and `optm` cases are within ~ 2 s.e. of FD, the blunt Kn 0.5 squash is unchanged (-1.4 ± 1.1 % vs FD; < 0.3 % of queries lowered). Source of these numbers: the docstring of `ensemble_backward.CELL_BASIS`.

Next to a surface. With the nearest-surface normals n_i of the slots (within a cell width, else 0) the carried value jumps across $v \cdot n = 0$ (a particle about to hit carries its imminent reward, one that just left does not), which no polynomial in v represents. Per cell: $n_c = \sum_i n_i / |\sum_i n_i|$ and the resultant $R_c = |\sum_i n_i| / \#\{i : n_i \neq 0\}$. Three fits (3.10) over the nodes with $v \cdot n_c > 0$

(*ahead*), $v \cdot n_c \le 0$ (*behind*) and all (*both*), the sides always guarded at $1.0\, m_g \le n$; a query with normal n_q reads

$$\Lambda(x_q, v_q) = \begin{cases} \Lambda_{\text{both}} & n_q = 0, \text{ or its side is empty, or } R_c < 0.9, \\ \Lambda_{\text{ahead}} & v_q \cdot n_c > 0, \\ \Lambda_{\text{behind}} & \text{otherwise.} \end{cases} \tag{3.12}$$

The cells with $R_c < 0.9$ (normals that disagree: corners, opposed faces, curvature at the cell scale; $\le 6\,\%$ of the flow on the optimiser's shapes) stay two-sided (`both`); the alternative of answering their queries with the per-query one-sided fit (the default without `both`: a static buffer of the complex cells' queries, `_Hybrid`, `ensemble_device.fallback_capacity`) is exact in the limit of every cell complex but inherits that fit's divergence on complicated geometry.

Why this form. The collision operator only sees the part of λ outside the collision invariants: for $\lambda \in \operatorname{span}(1, v, |v|^2)$ every four-term $\Lambda(v_i') + \Lambda(v_j') - \Lambda(v_i) - \Lambda(v_j)$ of (3.6) vanishes. What the pull-back needs is the cell population's velocity structure beyond the invariants, a property of the cell rather than of a neighbourhood of each query; the per-query weights added cost ($\propto k$) and a dependence on k (Kn 0.5 squash: $k = 32 \to 64$ moved $\mathrm{d}D/\mathrm{d}s$ by $+1.6 \pm 0.7\,\%$ over 12 seeds, flat from $k = 64$, where the fit spans the cell and its weights only range over $[\frac{1}{2}, 1]$). The per-cell fit has no k and costs C solves per substep instead of $(3 + 2K)N$. The guard is what makes it stable: a cell's values are mostly noise (on a squash snapshot the cell mean predicts held-out nodes at the noise floor), and an underdetermined quadratic in a sparse cell answers the partner ghosts with extrapolated noise that the backward recursion compounds; without the guard one seed in twelve read $+97\,\%$.

Measured (validation deck, section V): Kn 0.5 squash, 12 seeds, against the per-seed CRN FD: guarded per-cell $-1.9 \pm 2.3\,\%$, per-query $k = 64$ $-2.0 \pm 2.0\,\%$, $k = 32$ $-3.5 \pm 1.8\,\%$ ($f_{\text{num}} \times 4$); at production density ($f_{\text{num}} \times 1$, 16 seeds, 2026-10-12) the guarded per-cell fit reads $-3.7 \pm 0.8\,\%$, the same as the per-query $k = 32$ ($-3.4 \pm 0.8\,\%$), and the cubic-in-v basis `cell:cubicv:adapt1.5` $-0.7 \pm 0.8\,\%$ (paired $+3.1 \pm 0.2\,\%$ over the quadratic); a seed 985 s $\to$ 95 s (the backward 897 s $\to$ 13 s); every stored collisional gate within its paired error of the per-query runs; stable on the optimiser's complicated shapes where the per-query one-sided fit diverged. Shipped (2026-10-12; `jax_dsmc/cell_fit.py`): `ensemble_backward.CELL_BASIS` = `"cell:cubicv:adapt1.5:guard32:align0.9:both"` (2026-10-19: per query the highest order whose leverage is ≤ 32; unguarded until then) (the quadratic `cell:quad:...` before: at production density and the 300-step horizon it reads -0.7 / $-1.8\,\%$ against FD where the quadratic read -3.7 / $-5.0\,\%$, at the same cost; with the whole run captured it overshoots by $+1.9 \pm 0.7\,\%$); `"quadratic_full"` selects the per-query fit below. Options (evaluated, not shipped): other polynomial bases, leave-one-out order selection, pooled fits, and the Rao–Blackwellised partners, whose ghosts are the field's expectation over the VSS angle — the post-collision direction has mean $\mathbb{E}[\cos\chi]\,\hat{g}$ and covariance $\operatorname{var}(\cos\chi)\,\hat{g}\hat{g}^\top + \frac{1}{2}(1 - \mathbb{E}[\cos^2\chi])(I - \hat{g}\hat{g}^\top)$, $\mathbb{E}[\cos\chi] = \frac{\alpha-1}{\alpha+1}$, $\mathbb{E}[\cos^2\chi] = 1 - \frac{4\alpha}{(\alpha+1)(\alpha+2)}$, and six symmetric sigma points average any quadratic exactly.

The per-query fit (slopes of the shape adjoint; the host backward). For a query (x_q, v_q) in cell c, the nodes are the alive particles of c with the current node velocities, in the scaled coordinates $\tilde{p} = (x/\Delta x,\ v/v_{\text{scale}})$ (cell width per axis; a thermal speed). Take the $k = 32$ nearest nodes in that metric (`neighbors=32`; host `scipy.spatial.cKDTree`, device `lax.top_k`), with weights

$$\omega_n = \frac{1}{1 + (d_n/d_k)^2}, \qquad d_k = \max_n d_n, \tag{3.13}$$

and the design row of node n at offset $\delta = \tilde{p}_n - \tilde{p}_q = (\delta_x, \delta_v)$ (`BASIS = "quadratic_full"`, `LocalFit._design`, device `ensemble_device._design`):

$$\phi(\delta) = \big(1,\ \ \delta_x,\ \ \delta_v,\ \ \{\delta_{v,a}\delta_{v,b}\}_{a\le b},\ \ \delta_x^2\big) \in \mathbb{R}^{1+D+3+6+D_x}, \tag{3.14}$$

so any field quadratic in v (the adjoint of an energy-like reward) is represented exactly. (The host squares only the first spatial coordinate, $D_x = 1$; the device squares every spatial coordinate, $D_x = D$; identical for $D = 1$.) The weighted least-squares coefficients solve

$$\Big(\textstyle\sum_n \omega_n \phi_n \phi_n^\top + \rho\,\bar{d}\ \mathrm{diag}(0, 1, \ldots, 1)\Big)\beta = \textstyle\sum_n \omega_n \phi_n \lambda_n, \qquad \rho = \texttt{RIDGE_REL} = 10^{-8}, \tag{3.15}$$

where $\bar{d}$ is the mean diagonal of the normal matrix (a *relative* ridge on every column but the intercept). Then

$$\Lambda(x_q, v_q) = \beta_0, \qquad \nabla_v \Lambda = \beta_{1+D:\,4+D}/v_{\mathrm{scale}}, \qquad \nabla_x \Lambda = \beta_{1:\,1+D}/\Delta x. \tag{3.16}$$

`ensemble_device.field_query` returns $(\Lambda, \nabla_x\Lambda, \nabla_v\Lambda)$ (since 2026-09-18). The host `LocalFit(gradient=True` returns both slope blocks. The collision pull-backs consume only the value; $\nabla_x\Lambda$ and $\nabla_v\Lambda$ are the cotangents of the shape adjoint (section 7, `node_gradients`). A query in a cell with no valid node returns 0. The ridge is the relative 10^{-8} of (3.15) on both host and device, and it is not innocent (review section 5.12: the sequential gradient moves 3×10^{-3} between `RIDGE_REL` 10^{-10} and 10^{-8} on the $N = 256$ inlet).

4 Score terms and the parameter gradient

Two estimators share the forward. The *trajectory* (DiCE) estimator of `jax_dsmc/adjoint.py` consumes the per-particle log-likelihood accumulator `logh_acc` that the forward fills; the *ensemble* estimator of this note consumes none of it. This section lists both, so that "which score is accumulated" is unambiguous.

4.1 What the forward accumulates in `logh_acc` (trajectory route only)

1. NTC attempt count: $\log P(n_{\mathrm{att},c})$ of the stochastic rounding of (1.3) (`ntc.attempt_count`), and per attempt $\log h = \log p_{ij}$ on acceptance, $\log(1 - p_{ij})$ on rejection (`vss.log_h`).

2. Diffuse-wall accommodation Bernoulli (`boundary.diffuse_from_draws`):

$$\log p_{\mathrm{acc}} = \mathbf{1}[u \le \mathrm{acc}_f] \log \mathrm{acc}_f + \mathbf{1}[u > \mathrm{acc}_f] \log(1 - \mathrm{acc}_f), \tag{4.1}$$

   while T_f and u_f are *pathwise* there: the re-emitted velocity is the reparameterisation $v = \sqrt{k_B T_f/m}\,(z_t + \hat{n}\, r) + u_f$ of standard draws (z, r) (`sampling.diffuse_velocity_from_draws`), so `jax.grad` reaches T_f, u_f through the velocity and acc_f through the score only.

3. The ε-hit interval log-probability (`wall_adjoint.conditional_time`): the realised event history restricts the Gaussian flight time $\tau \sim \mathcal{N}(\Delta t, \varepsilon^2)$ to an interval $[\tau_{\mathrm{lo}}, \tau_{\mathrm{hi}})$ (last hit time, next hit time), and

$$\log p_\varepsilon = \log\Big[\mathrm{ndtr}\Big(\tfrac{\tau_{\mathrm{hi}} - \Delta t}{\varepsilon}\Big) - \mathrm{ndtr}\Big(\tfrac{\tau_{\mathrm{lo}} - \Delta t}{\varepsilon}\Big)\Big], \qquad \mathrm{ndtr} = \text{standard normal CDF}, \tag{4.2}$$

   with the sample itself re-expressed as the value-preserving conditional quantile so that its tangent is the conditional one (`corrected = stop_gradient(raw) + sampled - stop_gradient(sampl`). This is P3 section 4's random-time device; it is zero at $\varepsilon = 0$.

4. Inflow count (`inflow.inject`): $\log p_{\mathrm{count}} = \log \mathrm{frac}\,\bar{N}_f$ if the rounding went up, $\log(1 - \mathrm{frac}\,\bar{N}_f)$ otherwise, the only path by which n_ρ reaches the trajectory estimator.

4.2 What the ensemble estimator accumulates

All of the following weight a *carried value at the post-move state* λ^m (`Backward.lam`, `Births.values`, `Hits.values`); none of items 1–4 above is used, no ε-term appears, and no count score appears.

Diffuse walls: acc_f, T_f, u_f (all scored). The wall is a kernel and every recorded bounce h of every slot in every step (`WallEvents`, shape (K_{bounce}, N); newborn birth-step bounces included) contributes

$$\frac{\partial J}{\partial \theta_w} = \sum_h \lambda^m_{i(h)}\, \partial_{\theta_w} \log b_{f(h)}\big(v_h^{\mathrm{out}};\, \theta_w\big), \qquad b_f(v) = \begin{cases} \mathrm{acc}_f \, \dfrac{v_n\, e^{-m v_n^2/2k_BT_f}}{k_BT_f/m} \, \dfrac{e^{-m|v_t-u_f|^2/2k_BT_f}}{2\pi k_BT_f/m} & \text{diffuse branch,} \\ 1-\mathrm{acc}_f & \text{specular branch,} \end{cases} \tag{4.3}$$

with the branch read from `WallEvents.diffuse` $= \mathbf{1}[u \le \mathrm{acc}_f]$ (`examples/ensemble_couette.py::log_wall_k` assembled as $\sum_h$ `values`$_h \cdot$ `jax.grad`$(\log b)$). The weight is λ at the end of the move, not after this particular bounce, which is unbiased by the tower property whatever the number of later bounces in the step (review section 5.6). Note the difference from the trajectory route: there T_f and u_f are pathwise through the reparameterised draw and only acc_f is scored; here the realised re-emission is data and all three wall parameters enter through the kernel score.

Inflow sources: n_ρ, T, u of a reservoir (density score + intensity). With b_f the normalised inward flux density (`examples/ensemble_inlet_shock.py::log_flux_density`)

$$b_f(v;\theta) = \frac{(v\cdot n_f)\, \exp\!\big(-m|v-u_f|^2/2k_BT_f\big)}{Z_f}, \qquad Z_f = \pi^{3/2} v_{s,f}^4\, F(s_f), \qquad v\cdot n_f > 0, \tag{4.4}$$

$$\frac{\partial J}{\partial \theta} = \sum_{k=0}^{M-1} \sum_f \Big[\sum_{b\in\mathcal{B}_{k,f}} \lambda^m_b\, \partial_\theta \log b_f(v_b;\theta) \;+\; \partial_\theta \bar N_f(\theta) \cdot \frac{1}{|\mathcal{B}_{k,f}|} \sum_{b\in\mathcal{B}_{k,f}} \lambda^m_b \Big], \tag{4.5}$$

where $\mathcal{B}_{k,f}$ are the slots born through face f in step k that survived their birth flight (`Births`), v_b is the velocity *as sampled* (`born_v`), and $\bar N_f$ is (1.2) differentiated by `jax.grad` through `inflow.target_count`. The first term is P3 eq. `ensemble_inflow_gradient_left`/P3 eq. `sampled_left_score_` with P3's $N_L/m_{L,k}$ weight equal to 1 (every birth carries its own value, no interpolation); the second is the intensity derivative P3 states in words after P3 eq. `tangential_temperature_score_inflow` and leaves out of its formulation (N_L, N_R held fixed there). It is exact for the stochastically rounded count because $\mathbb{E}[N_f] = \bar N_f$ is smooth (review section 5.6, audit table). For the inlet shock every control acts through the Rankine–Hugoniot map, so ∂_θ of both b_f and $\bar N_f$ is taken by `jax.grad` through `inlet_shock_reservoirs`.

Initial condition. The same two terms with $\lambda_{0,i}$ (`Adjoint.lambda0`, the value at t_0 on the initially alive slots) against the initial Maxwellian's log-density and the initial count intensity (`ensemble_gradient` in `ensemble_inlet_shock.py`).

Collision-kernel parameters: `diam`, `omega`.

$$\frac{\partial J}{\partial \theta} = \sum_k \sum_{\text{pairs } p} W_p\, \Phi_p\, \partial_\theta \log q(v_{i_p}, v_{j_p}), \qquad W_p = \begin{cases} 1 & \text{realised pairs (sequential form),} \\ \tfrac{1}{2} w_{i\kappa} & \text{virtual pairs, (3.9),} \end{cases} \tag{4.6}$$

with $\partial_\theta \log q$ by `jax.grad` through `collide.species_params` (`examples/ensemble_relaxation.py`). Accepted pairs sample qff whatever the majorant, so no attempt-count term appears for a kernel parameter.

Not covered. The VSS angular parameter α acts pathwise through `scatter` and would need $\nabla_v \Lambda$ at the post-collision states; `field_query` returns it, no assembly exists (review section 5.0). Reparameterised (pathwise) source terms inside the ensemble sweep were measured on the P3 model and rejected as biased and 1.8× noisier (review section 5.9).

5 Births and deaths

Births. A slot born in step k is alive at t_{k+1}, collides in step k like any other, and so receives a value λ_b^m from the collision pull-back like any other. The backward then does two things with it (`backward`, block "2. sources"; device values are read off `Backward.lam` by `from_device`):

$$\texttt{Births.values}_b = \lambda_b^m = \lambda\big(x_{k,b}^m, v_{k,b}^m\big), \qquad \mu_b^{(k)} = 0, \tag{5.1}$$

i.e. the value at the newborn's post-move state (its face position plus the ξv_b first flight, after any birth-step bounces) is what the source score of (4.5) multiplies, and the slot carries nothing further back (it did not exist at t_k). P3 says the same: an injected particle's adjoint "contains its complete future effect" and there is no score for ξ because its law is parameter-free. A birth absorbed during its own first flight (`escaped`) is not alive at t_{k+1} and is absent from `Births`, so it contributes 0 to the score sum; the intensity mean in (4.5) is taken over the surviving births only (the code's `mean(b.values[on])`), whereas the P3-model route of review section 5.1 counts escaped births as zeros. The two agree when no birth escapes during its first flight; the difference is not measured on the production cases.

Deaths. A particle absorbed during the move of step k has $a_{k,i}^m = 0$, so $\lambda_i^m = 0$ (`lam = where(alive, lam, 0)`) and by (3.2) $\mu_i^{(k)} = 0$: the indicator of P3 eq. `ensemble_inflow_transport_adjoint`. Slots that are dead throughout carry 0 and are never fit nodes (`cell_nodes` marks them invalid). The device still *queries* dead slots (wasted work on capacity-padded runs, review section 5.12), which does not change any value.

6 Validation summary

Every number is copied from the review; "FD" is the same seeds' common-random-number finite difference unless marked "FD-only" (independent seeds). Sequential rows are review section 5.7; virtual rows are section 5.12 (staged nodes, $K = 8$, device; $S = 2$ unless the row says otherwise, the relaxation rows from its pull-back scan table); the last row is section 6.

Routes: *src+int* = source density score plus intensity term (4.5); *wall* = diffuse-wall kernel score (4.3); *kernel* = collision-kernel score (4.6); *value* = delta-method terminal vector (2.2).

control	route	pull-back	ensemble ± SE	arbiter ± SE	z
inlet `nrho`, N=128, M=4	src+int	sequential	1.375e5 ± 1.3e3	FD 1.377e5 ± 2.5e3	−0.08
inlet `nrho`, N=256, M=8	src+int	sequential	1.361e5 ± 1.9e3	FD 1.34e5 ± 2.3e3	0.71
inlet `temp`, N=128, M=4	src+int	sequential	1.345e5 ± 4.1e3	FD 1.373e5 ± 1.1e3	−0.77
inlet `temp`, N=256, M=8	src+int	sequential	1.364e5 ± 3.6e3	FD 1.374e5 ± 1.1e3	−0.32
inlet `temp`, N=128, M=4	src+int	virtual	1.302e5 ± 9.4e3	FD 1.368e5 ± 2.6e3	−0.8
inlet `nrho`, N=128, M=4	src+int	virtual	1.377e5 ± 3.1e3	FD 1.347e5 ± 6.0e3	+0.5
inlet `temp`, N=2048, Δt=0.5	src+int	virtual	1.361e5 ± 3.9e3	FD 1.386e5 ± 1.5e3	−0.7
inlet `nrho`, N=2048, Δt=0.5	src+int	virtual	1.367e5 ± 1.6e3	FD 1.375e5 ± 2.1e3	−0.4
Couette wall speed (ε=0.005)	wall	sequential	0.5916 ± 0.0097	FD 0.5999 ± 0.0062	−0.97
Couette wall speed (ε=0)	wall	sequential	0.5865 ± 0.014	FD 0.5933 ± 0.0093	−0.53
Couette T_{left}	wall	sequential	0.1634 ± 0.0088	FD 0.1804 ± 0.012	−1.2
Couette accommodation	wall	sequential	0.2917 ± 0.009	FD 0.2615 ± 0.017	1.7
Couette wall speed	wall	virtual	0.615 ± 0.030	FD 0.591 ± 0.020	+0.8
Couette T_{left}	wall	virtual	0.169 ± 0.018	FD 0.108 ± 0.031	+1.7
relaxation `diameter`	kernel	sequential	−0.663 ± 0.026	FD −0.6723 ± 0.012	0.32
relaxation `omega`	kernel	sequential	0.2824 ± 0.01	FD 0.2607 ± 0.014	1.2
relaxation `diameter`, S=4	kernel	virtual	−0.6753 ± 0.0070	FD-only −0.6800 ± 0.0034	(+0.7%, 0.6σ)
relaxation `diameter`, S=8	kernel	virtual	−0.6795 ± 0.0071	FD-only −0.6800 ± 0.0034	(+0.1%, 0.1σ)
relaxation `omega`, S=2	kernel	virtual	0.2730 ± 0.0039	FD-only 0.2706 ± 0.0049	(+0.9%, 0.4σ)
RH shock $d(\rho_2/\rho_1)/dM_1$, 300 steps	value	virtual	2.74 ± 0.66	FD 2.54 ± 0.17 / 2.17 ± 0.42	0.3

Further facts from the review used above: the ensemble's noise relative to the FD's (sequential form, section 5.7) is 0.5–0.8× for reservoir density, 0.7× wall temperature, 0.7× `omega`, 1.5× wall speed, 2.1× diameter, 3.3–3.8× reservoir temperature; the virtual form's per-seed sd is 7× below the sequential form's at 0.16 collisions/particle/step and 37–56× at 0.8 (section 5.12); the accommodation FD is a step function of a decision parameter and cannot arbitrate below $\sim 3\%$ at 384 seeds (section 5.6); the RH stationary slope 1.96 is a reference only; the pathwise VJP of the same RH functional had the wrong sign (−1.9e−3 vs FD +1.8e−3, `examples/shock/gradients.py`). Host and device agree to 10^{-8} on identical records for the sequential form and statistically for the virtual form (`tests/test_ensemble_device.py`); the virtual pull-back carries a quadratic invariant to 10^{-4} at the shipped ridge and 10^{-12} at zero ridge (`test_virtual_pullback_carries_an_invariant_exactly`). The shape oracle of section 8 is in `results/shape_planar_oracle.json`.

7 The shape adjoint (implemented 2026-09-18: `jax_dsmc/shape.py`)

The geometry θ (the centre and radius of an analytic body, or the vertex coordinates of a surface; normals and areas are rebuilt from it, the cut volumes and cell tables are not) enters the forward only through the move of each step, which under the RNG contract is a pure map

$$(x_k^{\text{post}}, v_k^{\text{post}}, \log p_{\varepsilon,k}, T_k, \mathcal{E}_k) = \mathcal{W}_\theta\big(x_k^{\text{pre}}, v_k^{\text{pre}}, \kappa_k\big), \tag{7.1}$$

of the previous post-collision state and the step's key κ_k (`key_for(base, k, STAGE_MOVE)`); T_k are the step's surface tallies and $\mathcal{E}_k$ its traced events (hit point x^h, incident velocity v^{in}, normal n, event time t^h of every recorded hit, `advance(trace=True)`). `shape.replay` re-runs $\mathcal{W}_\theta$ on the captured record and reports how well it reproduces the record (`replay_error`; the runners enforce it): any discrete disagreement (a particle alive in the replay and not in the record, or the reverse) reports 1 and is an error, and the continuous states must agree to 10^{-9} relative (`REPLAY_TOL` of the mesh runners) (the same arithmetic in another compilation; round-off is amplified $\sim 10^3$ by a few grazing reflections in a concave pocket, so the earlier 10^{-12} was too tight); `shape.gradient` takes `jax.vjp` of (7.1) with respect to θ at every captured step and sums

$$\frac{\mathrm{d}J}{\mathrm{d}\theta} \approx \sum_k \Big[\underbrace{\nabla_x \lambda_k^m \cdot \partial_\theta x_k^{\text{post}} + \nabla_v \lambda_k^m \cdot \partial_\theta v_k^{\text{post}}}_{\text{pathwise } x,v} + \underbrace{\partial_T J \cdot \partial_\theta T_k}_{\text{tally}} + \underbrace{w_k\, \partial_\theta \log p_{\varepsilon,k} + \sum_j w_{k,j}\, \nabla_n \log q(v_{k,j}^{\text{out}} \mid n_{k,j}) \cdot \partial_\theta n_{k,j}}_{\text{score}} \tag{7.2}$$

every $\partial_\theta(\cdot)$ being what the VJP of $\mathcal{W}_\theta$ delivers.

Cotangents. $\nabla_x \lambda_k^m$, $\nabla_v \lambda_k^m$ are the slopes of the per-cell fit (3.16) at each survivor's own post-move state (`field_query` now returns `(value, grad_x, grad_v)`; `node_gradients`). The fit is *one-sided*: only nodes with $v \cdot n > 0$ for the hit particle's last normal n are used, because next to a wall the post-move population splits into particles about to hit (value $\approx$ the hit's reward) and particles that just left, an $O(r)$ contrast a polynomial in v cannot represent and that is correlated with the distance to the wall (the approaching side is depleted within one flight of it) – the two-sided x-slope read that population mix, and the plane's shear gradient came out with the wrong sign (-0.012 ± 0.005 against FD $+0.017 \pm 0.003$). w_k is (2.5) summed over the step's hits; $\partial_T J$ is the objective's weight on the tally (a windowed drag: $f_{\mathrm{num}}/(W\,\Delta t)$ on the element rows' x-momentum). The diffuse re-emission velocity is `stop_gradient` inside the map: a kernel's parameters *and* its normal enter only through the score of the frame-independent density

$$q(v \mid n) = \frac{v_n}{s^2}\, e^{-v_n^2/2s^2}\, \frac{e^{-|v_t-u|^2/2s^2}}{2\pi s^2}, \qquad s^2 = kT_w/m,\ v_n = v\cdot n,\ v_t = v - v_n n \tag{7.3}$$

(`sampling.log_diffuse_density`); the specular branch and the hit point are pathwise (review section 5.9).

The hit-timing band term. The VJP differentiates a *fixed* hit set. A reward that changes at a hit – a velocity-dependent per-step reward, or a per-hit reward at the window's edges – also depends on which step a hit falls in, and moving the surface moves hits across the step boundary. With $\Delta J_{k,j}$ the change of the objective when hit j moves from the end of step k to the start of step $k+1$ ($\phi_k(v^{\mathrm{out}}) - \phi_k(v^{\mathrm{in}})$ for a state reward; $+r$ on the last window step and $-r$ on the step before the window for a per-hit reward; the value after either outcome is equal in distribution, so nothing else jumps), the boundary term is the density of hit times at the step end times $-\Delta J$, estimated from the hits within a band $\beta\,\Delta t$ of it:

$$c_{k,j} = -\frac{\Delta J_{k,j}}{\beta\,\Delta t}\,\mathbf{1}\big[t^h_{k,j} > (1-\beta)\Delta t\big], \tag{7.4}$$

placed on the traced event time so that $\partial_\theta t^h$ is exact for any body. It is the value-preserving analogue of the ε hit-timing score of (4.2) with no forward regularisation. On the plane's windowed v_y^2 the pathwise term alone was $+0.82 \pm 0.23$ against FD $+0.05 \pm 0.15$; the band term is -0.43 ± 0.02 ($\beta = 0.2$ and 0.4 agree to 1 %). A per-hit reward's term lives at the two window edges and has zero mean on a steady window (the cancelling edge effect of gate G3a), so `examples.ensemble_mesh` reports it and leaves it out of the estimate.

Estimating $\nabla_x \lambda$: the one-sided, pooled and along-flight forms. The cotangent $\nabla_x \lambda_k^m$ is the weak link of (7.2): next to a surface the post-move population splits into particles about to hit (value $\approx$ the hit's reward) and particles that just left, an $O(r)$ contrast a polynomial in v cannot represent, and the per-hit signal of a translated wall is a window-end quantity far below the fit's slope noise. Four estimators are implemented and compared against replica-exact gradients (review section 7.6, item 5):

1. *one-sided*: the fit (3.16) over the nodes with $v \cdot n > 0$ for the query's last hit normal n (`node_gradients(direction=)`); the general rule for any query uses the nearest surface within one cell width (`mesh.geometry.nearest_normal`, `backward_virtual(normals=)`);
2. *pooled*: the k nearest one-sided nodes drawn from the 3^D block of cells around the query's cell (`pool=1`, $k = 96$; `ensemble_device.pooled_tables`);

3. *along-flight*: collisionless free flight makes $V(x - sv, v)$ the value of the same particle delayed by s, so

$$v \cdot \nabla_x V(x,v) = -T(x,v), \qquad T(z) = \mathbb{E}\Big[\sum_{k'>k} \sum_j \tau_{k',j} \,\Big|\, z_k = z \Big], \qquad \tau_{k,j} = c_{k,j} \text{ of (7.4)}, \tag{7.5}$$

   T being the value of the *timing rewards* – the change of the objective per unit delay of the trajectory – carried by a second value-form backward with `hit_reward` $= \tau$ (`Replay.timing_reward`). The cotangent then replaces the fitted component along v by the carried value, $\nabla_x\lambda \leftarrow \nabla_x\lambda - \frac{(v\cdot\nabla_x\lambda)+T}{|v|^2}\, v$ (`shape.along_flight`), and keeps the fitted perpendicular part. With collisions the identity acquires the collision operator's term ($v \cdot \nabla_x V = -\text{rate} - Q^*V + \partial_t V$); it is neglected here (the cases run at $\leq 0.3\,\%$ collisions per particle-step);

4. *pooled + along-flight*: (2) for the perpendicular part, (3) along v.

Since τ is nonzero only at band hits, the carried T is a sparse sample of its expectation: unbiased, but the per-hit reward at the window's two edges is what a per-hit objective's T consists of, while a velocity-dependent per-step reward has band hits at every step. Measured (review section 7.6, item 5; replica-exact gradients on the plane and the production runs): the along-flight form removes the fitted estimator's bias on the windowed v_y^2 ($-0.372 \pm 0.067 \to -0.188 \pm 0.040$ against the truncated FD -0.149 ± 0.037) at half the variance; pooling lowers the per-seed sd 4–7× on the circle radius but on its own keeps a bias (-0.20 ± 0.11 on the v_y^2 x-part); the combination (4) is the default of the gradient runners. What none of them resolves is the x/v-part of a translated flat wall: on the shear probe the exact value is $+0.0032 \pm 0.0006$ and the pooled / along-flight forms read 0 (pooling smooths a slope a tenth of the per-particle noise to zero; the realised T is a sparse sum of window-edge band hits whose sample SE understates its uncertainty), while the one-sided fit is consistent at 3× the noise. Radius-type controls are FD-consistent to a few per cent because their fixed-hit-set tally term is exact; translation-type controls of flat walls are not resolved by the fitted or the value-based forms here. *Decision (open issue):* shape controls are therefore restricted to normal displacements of curved surfaces, which is sufficient for the first optimisation case (a circle deformed toward an airfoil-like shape, and the sphere analogue); tangential translations of flat walls are not supported as controls.

Stratifying the fit by the hit-ahead class. The one-sided rule separates the two populations exactly at a flat wall and not at a curved one: a particle leaving a cap's tilted tangent plane can still be heading for the flat face below it, so the pool of a reflected query mixes in nodes that carry their next hit's reward, and the fitted $\partial\lambda/\partial v_x$ reads the mixture's $d(2mv_x)/dv$ times the hit probability. On the specular capsule (review section 7.9) this is a $-27\,\%$ bias of the total against the CRN FD, entirely in the $\nabla_x\lambda$, $\nabla_v\lambda$ parts, whose true value is zero: for a convex body in a collisionless flow a reflected particle never re-hits, and the hit-reward (tally) term alone reproduces the FD seed by seed. The record holds every particle's future, so the class is exact: the nodes of a query are the particles of its own class,

$$\kappa_k^m = \mathbf{1}\{\exists\, k' > k : \text{ particle } m \text{ hits the body at step } k'\}, \qquad \text{nodes of } (k,m) = \{\, m' : \kappa_k^{m'} = \kappa_k^m \,\} \tag{7.6}$$

(`shape.hit_ahead`, `node_gradients(klass=)`). For the collisionless convex bodies the post-hit class has $\lambda \equiv 0$ and the $\nabla\lambda$ parts vanish; in a collisional gas the class-conditional fit is the smooth part of the value gradient, the class boundary (a hit-set change) being what the silhouette and timing terms carry. Specular capsule, four CRN seeds: $0.964 \pm 0.037 \to 1.356 \pm 0.011$ against the FD 1.35×10^{-20} ($z = -11 \to +0.08$), the x/v parts $-3.9 \to +0.05 \pm 0.6 \times 10^{-21}$.

Representation. The shape is optimised as a C^1 level set $\phi(x;\, c) = \sum_j c_j B_j(x)$ (uniform cubic B-splines on a grid finer than the DSMC grid; body $\phi < 0$, normal $n = \nabla\phi/|\nabla\phi|$ into the flow), not as polygon vertices. For a flat-normal polygon the Hadamard derivative carries a delta at every vertex (the normal jumps there); under uniform scaling that corner term is the whole $\mathrm{d}F/\mathrm{d}r$. On a C^1 surface the hit normal varies smoothly with the shape and the pathwise derivative of the replayed wall map is complete for specular walls; a C^0 interpolant would merely move the jumps to the level-set cell faces. The hit time is the root of $\phi(x + tv) = 0$, found by bracketing and bisection and finished by one Newton step with the bracket held constant, so that $\partial t/\partial c_j = -B_j(x_{\mathrm{hit}})/(v\cdot\nabla\phi)$ (the implicit-function derivative; the bisection alone has none). The only non-pathwise term that survives is the silhouette term of a diffuse body (a near miss becoming a grazing hit), on the set $\min_s \phi(x + s\hat{v}) \in (0, \varepsilon)$; it vanishes for specular walls.

The silhouette band term. Let $\phi_{\min}(\omega) = \min_s \phi(x + s\hat{v})$ be the closest approach of a particle's undeflected flight to the surface (< 0: it hits, with that chord depth; > 0: it misses by that distance). A displacement of the surface changes the outcome of the particles with $|\phi_{\min}| < |\delta\phi|$; the change of the objective is $V_{\mathrm{hit}} - V_{\mathrm{miss}}$, the value of the realised outcome being the particle's reward-to-go and that of the virtual one (a straight flight for a hit, a re-emission from the closest point under the wall model for a miss) the fitted adjoint at the virtual post-move state plus the virtual hit's reward. With $\rho(0)$ the density of $\phi_{\min}$ at zero,

$$\frac{\mathrm{d}J}{\mathrm{d}\theta}\Big|_{\mathrm{sil}} = -\,\mathbb{E}\Big[\rho(0)\,(V_{\mathrm{hit}} - V_{\mathrm{miss}})\,\frac{\partial\phi_{\min}}{\partial\theta}\Big] \approx -\frac{1}{2\varepsilon_b}\sum_{|\phi_{\min,i}|<\varepsilon_b} (V_{\mathrm{hit},i} - V_{\mathrm{miss},i})\,B_j(x_i^*)\,\frac{\partial c_j}{\partial\theta},$$

the band on both sides of the surface (window $2\varepsilon_b$) estimating $\rho(0)$, and $\partial\phi_{\min}/\partial c_j = B_j(x^*)$ by the envelope theorem at the frozen closest point x^*. Every passing is counted once: a miss in the step whose move contains x^* (strictly inside – a particle starting there has just left the surface), a hit in the step it hits, with $\phi_{\min}$ taken along the line within the search reach. For a specular wall $V_{\mathrm{hit}} \to V_{\mathrm{miss}}$ at grazing incidence and the term is $O(\varepsilon_b)$; for a diffuse wall it is the $O(1)$ term of the silhouette, and on constructed rays with $\lambda = 0$ it reproduces (new hits per unit displacement) $\times$ (absorbed momentum) to 0.04 % at $\varepsilon_b = 0.02$.

Grazing regularisation. The pathwise derivative of a hit carries $\partial t/\partial\theta = -\,\partial_\theta\phi/(v\cdot\nabla\phi)$; over the ray measure $\mathrm{d}b$ its mean converges but its variance diverges logarithmically at grazing incidence (one hit at $|v\cdot n|/|v| = 3\cdot 10^{-3}$ was 15 % of a window's tally sum). The denominator is evaluated as $\max(|v\cdot\nabla\phi|,\ \epsilon\,|v|\,|\nabla\phi|)$ in the tangent only (the primal hit is exact): the per-hit weight is bounded by $1/\epsilon$ and the bias is $O(\epsilon)$ – measured on the circle's radius tally term over 8 CRN seeds, the seed sd falls from 8.4 % to 4.2 % ($\epsilon = 0.03$), 2.9 % (0.1), 2.1 % (0.3) with paired biases $+0.5 \pm 1.8$, -1.2 ± 2.3, -7.4 ± 2.4 %. The diffuse normal score has the same tail on the emission side: $\partial_n \log q = v_{\mathrm{out}}/v_n - v_n v_{\mathrm{out}}/s^2 + \ldots$ with v_n Rayleigh, so $\mathbb{E}[(v_t/v_n)^2] = \int \mathrm{d}v_n\,(v_n/s^2)\,|v_t|^2/v_n^2$ diverges logarithmically at $v_n \to 0$ (the flux-weighted cosine law puts mass ϵ^2 below $v_n/|v| = \epsilon$). The derivative of $\log v_n$ is evaluated at $\max(v_n,\ \epsilon|v_{\mathrm{out}}|)$ with the same ϵ (primal exact); the bias of the clipped part is $\int_0^{\epsilon s}(v/s^2)(1/v - 1/\epsilon s)\,\mathrm{d}v\cdot|v_t| = \epsilon|v_t|/2s$, i.e. about $\epsilon/2$ of the v_t/v_n part.

Hybrid form of the diffuse normal term. With the per-hit reward $r(v_{\mathrm{out}})$ exact, the normal dependence of a diffuse hit splits as

$$\frac{\mathrm{d}}{\mathrm{d}n}\,\mathbb{E}\big[r(v_{\mathrm{out}}) + V(v_{\mathrm{out}})\big] = \mathbb{E}\Big[\frac{\partial r}{\partial v_{\mathrm{out}}}\cdot\frac{\partial v_{\mathrm{out}}}{\partial n}\Big] + \mathbb{E}\big[V(v_{\mathrm{out}})\,\partial_n \log q(v_{\mathrm{out}} \mid n)\big],$$

the first term reparameterised through $v_{\mathrm{out}} = \sigma\,(r\,n + z_1 t_1(n) + z_2 t_2(n)) + u$ at fixed draws (weights bounded by $m|v|\,|\partial n/\partial\theta|$), the second the score weighted by the value-to-go *after* the

hit, which needs no fitted derivative of λ. Because the tangential law is isotropic any smooth frame field $t_{1,2}(n)$ gives the same pushforward, and the derivative is taken in the frame parallel-transported from the primal one, $t_1(n) = (a-(a\cdot n)n)/|\cdot|$ with a the frozen primal tangent, whose derivative $-(a\cdot\delta n)\,n$ is bounded; the incident-velocity frame of the sampler has $\partial t_1/\partial n \sim 1/|v_{t,\mathrm{in}}|$ at normal incidence, a log-divergent variance for nothing.

What the terms are on the analytic circle. For a specular cylinder of radius r in free-molecular flow the fixed-hit-set part of dF/dr is the whole of F/r: with impact parameter b, $\cos\varphi = \sqrt{1-b^2/r^2}$ and $F \propto \int_{-r}^{r} \cos^2\varphi\,\mathrm{d}b = 4r/3$, the derivative at fixed rays $\int 2b^2/r^3\,\mathrm{d}b = 4/3$ equals F/r while the new grazing rays contribute $\cos^2\varphi = 0$. Measured (SPARTA circle at $f_{\mathrm{num}} \times 4$, 700 warm-up steps, 200-step window): the tally term is $1.27 \pm 0.02 \times 10^{-20}$ against $F/r = 1.24\times10^{-20}$; the flow-field feedback of the blockage (FD $- F/r \approx 0.15\times10^{-20}$) is what the $\nabla\lambda$ terms carry, and the x-part carries most of the estimator's noise (see the review, section 7.6).

7.1 The shape optimiser (`examples/mesh/optimize.py`)

The optimiser has two separate parts. The *gradient estimator* is the ensemble adjoint of this note. The *update rule* is plain gradient descent (`gd`) or Adam. Adam is used only as a step rule on the adjoint's gradient: it never evaluates the objective and does not estimate the gradient itself.

Design variables and objective. The controls are the level-set coefficients $c \in \mathbb{R}^{m_1\times\cdots\times m_D}$ of $\phi(x;c) = \sum_j c_j B_j(x)$ (spacing h_ϕ; $h_\phi = 0.0625$ against the DSMC cell $h = 0.5$ for the coarse circle). The objective $J(c)$ is the windowed drag: the x-momentum given to the body over the last $W = 200$ of $700 + 300$ steps, $J = \frac{F_{\mathrm{num}}}{W\Delta t}\sum_{k\in\mathrm{window}}\sum_{\mathrm{hits}} m\,(v^{\mathrm{in}} - v^{\mathrm{out}})_x$. The body is specular in the `optc` study. Every iterate runs the same seeds $s = 1,\dots,S$ (common random numbers across iterates, so the objective history is comparable). Its gradient is the seed mean $g = \frac{1}{S}\sum_s \widehat{\nabla_c J}_s$ of (7.2) (pathwise x, v and tally parts, the score part and the silhouette part, `pathwise + score + silhouette` in `optimize.seed_terms`; the `along_pool` cotangent). The silhouette part is zero for these specular bodies (`--silhouette-band 0`); before 2026-09-28 the optimiser omitted it, which would have mattered only for a diffuse body.

One iterate.

1. Build the cut-cell mesh of $\phi(\cdot;c)$ on the host: classify cells, compute cut volumes, and detect split cells.
2. Run S forward + ensemble-backward solves and average: g and $\bar{J}\pm$SE.
3. *Band and H^1 smoothing.* With $M_j = \mathbf{1}[|\phi(x_j)| \le b + h_\phi]$ the coefficients whose knot is within the band $b = 1$ of the surface,
$$d = M \odot \big(G_\sigma * (M \odot g)\big), \tag{7.7}$$
G_σ a Gaussian of $\sigma = 2$ knots. This is a cheap Sobolev (H^1-type) gradient that removes the knot-scale noise of the estimator.
4. *Adam only:* $m \leftarrow \beta_1 m+(1-\beta_1)d$, $v \leftarrow \beta_2 v+(1-\beta_2)d^2$, $d \leftarrow \hat{m}/(\sqrt{\hat{v}}+\epsilon)$ with the usual bias corrections ($\beta_1 = 0.9$, $\beta_2 = 0.999$), then d is smoothed again by (7.7). The elementwise normalisation undoes the smoothing: at $k = 0$ it is exactly $\mathrm{sign}(d)$, and unsmoothed it grew overhangs.

5. *Fixed-size step:* $\eta = s_{\max}/\max_j |d_j|$ and $\tilde{c} = c - \eta d$, $s_{\max} = 0.05$. Since ϕ is kept a signed distance ($|\nabla\phi| = 1$), a coefficient change is a normal displacement: the surface moves outward by $V_n = -\delta\phi = \eta\, d$, so $s_{\max}$ is the largest boundary displacement per iterate (a tenth of a DSMC cell). There is no line search and no acceptance test; the noisy $\bar{J}$ is recorded, not used.

6. *Reinitialise* $\tilde{c}$ to the signed distance of its own zero set (closest-point projection of the band knots, distance to the projected cloud for the rest; `levelset.reinitialise`).

7. *Restore the area:* find the scalar γ with $A(\tilde{\phi} + \gamma) = A_0$ by bisection on $[-4h_\phi, 4h_\phi]$ (40 steps; A = sub-sampled smoothed-Heaviside body area, `levelset.body_volume`), and set $c_{k+1} = \tilde{c} + \gamma$.

Why the uniform shift is a projection. The area constraint is not imposed on the gradient. The step is taken unconstrained, and the area is then restored by a uniform shift of ϕ, which is a uniform normal displacement $-\gamma$ of the whole surface. To first order in the step, $\delta A = \oint_\Gamma V_n \,\mathrm{d}s$ for $|\nabla\phi| = 1$, so

$$A_0 = A\big(\phi - \eta d + \gamma\big) \approx A_0 + \oint_\Gamma (\eta d - \gamma)\,\mathrm{d}s \;\Rightarrow\; \gamma \approx \eta\,\langle d\rangle_\Gamma, \qquad V_n = \eta\big(d - \langle d\rangle_\Gamma\big),$$

with $\langle\cdot\rangle_\Gamma$ the perimeter mean. The net displacement is the $L^2(\Gamma)$ projection of the (smoothed, possibly Adam-normalised) step onto the tangent space $\{\oint V_n = 0\}$ of the constraint. The shift makes the constraint hold exactly, not only to first order, so the method is a projected-gradient (retraction) method, and the projection acts on the step, not on the raw gradient. Two consequences:

- For drag the gradient is dominated by its mean. A smaller body has less drag, so g mostly says "shrink everywhere". The shift removes that part, and what is left is the redistribution of area, not a loss of it.
- The Adam moments see the unprojected d, and $\max_j |d_j|$ normalises before the projection. The realised largest displacement is therefore $\le s_{\max} + |\gamma|$, and the history records γ (`area_shift`) for every iterate.

The predicted first-order change $-\eta\, g\cdot d$ is recorded as well. It is computed before the projection and is therefore an upper bound on the decrease.

The differentiated alternative (`--area-mode lagrangian`, 2026-10-12). The projection removes, after the fact, a component the step was sized for: in the long-box free-molecular Adam runs the shift was $\gamma \approx -0.010$ to -0.016 per iterate against $s_{\max} = 0.05$, a quarter of the step, and the Adam moments accumulate that removed component. The Lagrangian mode puts the area into the objective,

$$\mathcal{L} = D + \nu\,(A - A_0) + \tfrac{\mu}{2}(A - A_0)^2, \qquad \nabla_c\mathcal{L} = \nabla_c D + \big(\nu + \mu(A - A_0)\big)\nabla_c A,$$

with $\nabla_c A$ the exact gradient of `body_volume` (smoothed Heaviside; $\sum_j \partial A/\partial c_j = -$ perimeter, checked to 10^{-9} on the circle), so smoothing, band restriction and Adam act on the full gradient; only the face margin (`keep_inside`) stays a post-step constraint. The multiplier is updated after each step, $\nu \leftarrow \nu + \mu(A_{k+1} - A_0)$ (augmented Lagrangian), from $\nu_0 = -p$ with p the squash runs' area price ($\mathrm{d}D_{\mathrm{ref}}/\mathrm{d}s \,/\, \mathrm{d}V/\mathrm{d}s$ of the free-molecular ellipse at $s = 0.35$, at the run's density), and $\mu = 100\,p/A_0$ (a 1 % area error pulls as hard as p). The squash cost $D - pA$ alone (fixed ν, $\mu = 0$) is unusable for a free shape: $p = 2.4 \times 10^{-21}$ is 4× the circle's own $\mathrm{d}D/\mathrm{d}A = 6.0 \times 10^{-22}$ under uniform growth ($R = 3$), and with $D \propto R$ against $A \propto R^2$ the body would grow without bound; the quadratic term and the multiplier update hold A at A_0.

7.2 Status of the shape work (2026-09-24)

Forward on embedded bodies (validated).

- Surfaces: SPARTA line/triangle files, analytic circle/sphere, and the C^1 level set, embedded in the Cartesian grid.
- Cut cells: flow volumes by stratified sub-sampling, used in NTC.
- Split cells: SPARTA-style sub-cells for disconnected flow regions of one cell, each with its own volume and collision table (PSEUDOCODE 4.5).
- Tracking: the SPARTA move loop (section A.5) with an unbounded-in-practice bounce count (lane tracker, 32 passes).
- Gated against the rebuilt serial SPARTA on the circle and sphere cases (fill curves, surface loads, fields).

Shape gradients validated against CRN finite differences.

- Level-set circle radius: $z = -1.4$; per seed it agrees with the analytic body to 10^{-4}.
- $\cos 2\theta$ normal mode, coarse (grid 1, $f_{\text{num}} \times 4$, 4 seeds, before the hit-ahead class): $z = +0.7$ and -0.2. See "Open" for the fine-grid result.
- Sphere radius (8 seeds): $z = -1.71$.
- Specular capsule with the hit-ahead fit: $z = +0.08$.
- Diffuse plate (silhouette + hybrid normal term): 1.191 ± 0.068 against FD 1.17–1.34×10^{-20}, $|z| \leq 1.6$ at five FD steps.
- Lane tracker against the old stencil tracker: the gradients agree to 2.3×10^{-15} at production scale.

Open.

- **The hit-ahead class biases the $\cos 2\theta$ gradient** (2026-09-25). On the fine circle (grid 2, $f_{\text{num}} \times 1$, 32 seeds) the adjoint is -2.386 ± 0.005 against CRN FD -1.592 / -1.599×10^{-20} ($z \approx -120$). The tally term alone (-1.597) matches the FD seed by seed; the excess is in the fitted $\nabla_v \lambda$ (-0.68) and $\nabla_x \lambda$ (-0.11) parts. The coarse configuration on the current code reproduces the old artifact exactly with `value_class="none"` and shows the same excess with `"hit_ahead"`, so the stratification (7.6) is the cause. The suspected mechanism (slot refills: the departed particle inherits the newborn's later hits) is real, since 30 % of the coarse circle's hit-ahead labels come from a later occupant, but it is *not* the cause (2026-10-02, `scratch/shape_audit/`). With the class stopped at the slot's next birth, the coarse gate's v part grows from -6.0 to -10.8×10^{-21}; with no class it is $+0.4$, and the paired gradient − FD is $+0.007 \pm 0.071 \times 10^{-20}$ (4 seeds). The class is biased by construction: a fit within each class drops the $\nabla P\,(\lambda_1 - \lambda_0)$ part of $\nabla\lambda = P\nabla\lambda_1 + (1-P)\nabla\lambda_0 + \nabla P\,(\lambda_1 - \lambda_0)$, P the probability of a later hit. Squash runs use `value_class="none"`; the `optc` study ran with the class on. Slides: `docs/math/shape_validation.tex`.
- The diffuse circle meets the criterion against the noisy FD but is 9 % low against the precise one ($z = -2.3$; silhouette 61 % of the total). More seeds and a band sweep are pending.

- The Kn ≈ 0.5 gates are not run: fits at low occupancy (G4) and the cut-volume term (G5).
- The collisional term of the along-flight identity (7.5) is neglected.
- Translations of flat walls are not supported as controls.

Memory and robustness.

- Records store wall events compactly (only slots with an event). The replay keeps only what the VJP needs.
- The full forward + backward chain peaks at ≈ 64 kB per particle slot (was 116).
- The forward is $2\times$ faster than the fixed four-pass tracker.
- A fixed ≈ 300 MB spike of the shape VJP remains.

Failures the optimised shapes exposed, each now fixed. Concave notches produced multi-bounce traps (5–7 specular hits per step), a notch that split a cell into two flow pockets, Adam overhangs, and grazing multi-bounce round-off that tripped a 10^{-12} replay check. Not fixed: the Adam bodies of the `optc` study reach the inflow and outflow faces (below), where entrants are born inside the body (`inflow_shortfall`).

Optimisation study.

- Earlier result: 8 GD iterates on the circle cut drag by 7.4 % at fixed area (review 7.7).
- Current study (`examples/mesh/cluster`, group `optc`): the coarse circle, $S \in \{1, 2, 4, 8\}$ seeds per iterate $\times$ {`gd`, `adam`}, a target of 250 iterates each (`study.py --iterations 250`).
- Cost: 270–290 s per seed per iterate on one 24 GB RTX PRO 6000 MIG slice (seeds run in turn; $S = 8$: median 35 min per iterate, p90 40 min).
- Outcome (`examples/mesh/cluster/study_results.json`, `figures/optc_drag.png`): S1 Adam completed 250 iterates; the others stopped at the 24 h limit, a host out-of-memory kill (S8 GD) or a cancellation, and S2 / S8 Adam failed when the body crossed the inflow face. Drag re-measured on four held-out seeds agrees with each run's own CRN value within 1 %: GD -14 to -19 %, Adam -35 to -42 % against the circle. The Adam shapes pass through the outflow face from iterate 43 (S1, S2, S4) and reach the inflow face; the area constraint counts only the area inside the box, so these reductions are not those of an isolated body. A body-to-face clearance constraint is needed before they are quoted.

8 The shape extension as first planned (superseded by section 7)

This paragraph is the plan review section 6 assigned to a wall-position control θ before the terms were built; kept for the code comments that point at it. (i) *Hit set*: the ε-hit interval log-probability $\log p_\varepsilon$ of (4.2), differentiated with respect to the wall position instead of being ignored, times the carried value at the post-move state, $\sum_i (\lambda_i^m + r_{k,i}^{\mathrm{hit}}/\mathcal{N})\, \partial_\theta \log p_{\varepsilon,i}$, the per-hit reward r_k of section 2 riding along; the collisionless planar oracle (`examples/shape_planar_oracle.py`) recovered 0.588 ± 0.008 against the analytic 0.5917 with this value-weighted form, where the frozen-hit pathwise gradient gave 0. (ii) *Reflected position*: a replayed wall-map VJP with the

cotangents $(\nabla_x\lambda, \nabla_v\lambda)$ from (3.16) at the post-hit state, $\nabla_x\lambda\cdot\partial x'/\partial\theta$ with $\partial x'/\partial\theta = 2$ for a specular plane, requiring `field_query` to return the x-slopes it already computes. (iii) An *edge-band score* for what the VJP cannot see: on a flat-normal polygon every element's normal is constant under a scaling of the body, so the whole dF/dr sits in *which* element a ray meets ($O(1/E)$ jumps at $O(E)$ vertices), and a diffuse body has an $O(1)$ silhouette term (a grazing hit re-emits thermally; for a specular body it vanishes since $v_{\text{out}} \to v_{\text{in}}$ at grazing incidence). For hits within a band ε_b of an element edge and near-misses within ε_b in impact parameter, the alternative outcome is evaluated through the fit and the term is $(\Lambda_{\text{alt}} - \Lambda_{\text{actual}})\,\partial_\theta(\text{edge position})/\varepsilon_b$, value-preserving like the conditional time and reported separately. (iv) The cut-volume route: the NTC weight $2n_{\text{att},c}/N_c$ of a cut cell depends on $V_c(\theta)$ through (1.3), entering as $-\partial_\theta \log V_c$ times the substep's four-term sum in (3.6) (the `pair_score` slot); built only if the Kn ≈ 0.5 probe shows it matters. There is *no* cell-membership term: the cells are fixed (the body is embedded in the Cartesian grid, SPARTA-style), so a shape change never moves a cell face. Arbiters, in order: an embedded specular plane in a fixed box (position as control) against ensemble FD over independent keys, $|z| \le 2$ at two Δt; then the analytic circle's radius (free-molecular $dF/dr = F/r$ for a cylinder, $2F/r$ for a sphere, in an unbounded stream); then the 50-gon through the edge-band score.

A The DSMC forward solver, for CFD readers

This appendix writes out what one forward run of the code does, in the order it does it. It uses only the forward code (`simulate._step` and the modules it calls). Equation numbers are *A.n*.

A.1 What is being solved, and how it differs from CFD

DSMC solves the Boltzmann equation for the one-particle distribution $f(x, v, t)$ of a dilute gas,

$$\partial_t f + v \cdot \nabla_x f = Q(f, f), \tag{A.1}$$

where Q is the binary-collision operator. It is used where continuum closures (Navier–Stokes) fail, i.e. when the Knudsen number $\text{Kn} = \lambda/L$ (mean free path over body size) is not small. The SPARTA circle case is $\text{Kn} \gg 1$, close to free-molecular.

- **Unknowns.** DSMC does not discretise (A.1) on a grid. It samples f with N *simulator particles* (x_i, v_i), each standing for F_{num} real molecules.
- **Operator splitting.** Over one time step the equation is split into free flight ($\partial_t f + v\cdot\nabla f = 0$, solved exactly by moving particles and applying the boundary conditions) and a spatially homogeneous collision step ($\partial_t f = Q$, solved stochastically in each cell).
- **The grid.** It is only a partition used to (i) choose collision partners that are close to each other and (ii) average particle data into fields. There are no fluxes, no reconstruction and no linear solves.
- **Resolution requirements.** The step must be below the mean collision time and the cell below the mean free path, with $\gtrsim$ 10–20 particles per cell for the collision statistics.
- **Error.** Every macroscopic output is a moment of the particles. Its error is statistical, $O(1/\sqrt{\text{samples}})$, on top of the $O(\Delta t^2, \Delta x^2)$ splitting and cell errors. This noise is why the gradients in this note are expectations estimated over seeds.

The state is a fixed-capacity array of N slots: $x \in \mathbb{R}^{N\times D}$ ($D = 1$, 2 or 3 spatial dimensions; 2 or 3 with an embedded body), $v \in \mathbb{R}^{N\times 3}$ (always three velocity components, as in SPARTA), and an alive mask. Particles enter and leave by reviving and killing slots. Every array has a static shape, which is what lets the whole step be JIT-compiled and differentiated.

A.2 One time step

1. **Move** every alive particle for Δt, handling every wall, surface and outflow event on its path (section A.5).
2. **Inject** new particles through the inflow faces (section A.4); they are moved for a random fraction of Δt.
3. **Sort** particles by collision cell: compute each particle's cell index, `argsort` by index, per-cell counts N_c and start offsets (`sort.build_table`). Dead slots go to a sentinel cell.
4. **Collide** within every cell, independently and in parallel (section A.7). Collisions change velocities, not positions.
5. **Sample** (when requested): accumulate the cell moments and the surface tallies (section A.8).

Every stage draws its random numbers from its own key $\texttt{key_for}(\kappa, k, s)$ (1.5), so a run is a deterministic function of the base key κ and the parameters. This is what makes common random numbers and exact replay possible.

A.3 Initial state

For a box without a body, $N_0 = nV/F_{\text{num}}$ particles are placed uniformly in the box. Each gets a Maxwellian velocity

$$x_i = x_{\text{lo}} + U_i \odot (x_{\text{hi}} - x_{\text{lo}}), \qquad v_i = u + \sqrt{kT/m}\,\xi_i, \qquad U_i \sim \mathcal{U}[0,1]^D,\ \xi_i \sim \mathcal{N}(0, I_3) \tag{A.2}$$

(`initialize.initial_state`). With a body, SPARTA's `create_particles` draws the same way and rejects every sample that lands inside the body: accept/reject on position, with the count scaled by the flow volume. The circle, sphere and optimisation cases, like SPARTA's reference inputs, instead start from an *empty* box and fill from the inflow (≈ 700 warm-up steps), so no body-aware initial fill is needed.

A.4 Inflow boundary (emission)

Behind every inflow face f is a reservoir at (n, T, u). With $v_s = \sqrt{2kT/m}$ (most probable speed) and speed ratio $s = u \cdot n_f / v_s$, the expected number of simulator particles entering per step is (1.2), with the one-sided flux

$$F(s) = \frac{e^{-s^2} + \sqrt{\pi}\, s\,(1 + \operatorname{erf} s)}{2\sqrt{\pi}}. \tag{A.3}$$

The realised count is the stochastic rounding $\lfloor \bar{N} \rfloor + \mathbf{1}[U < \bar{N} - \lfloor \bar{N} \rfloor]$. Each entrant gets:

- a uniform position on the face;
- tangential velocity components that are Gaussian about the stream, $u_t + \sqrt{kT/m}\,\xi$;
- a normal component $v_n = v_s\, y$ drawn from the *flux-weighted* density

$$p(y) \propto y\, e^{-(y-s)^2}, \qquad y \geq 0. \tag{A.4}$$

Faster particles cross a face more often, hence the factor y.

The classic sampler (Bird; SPARTA `fix_emit_face.cpp`). Acceptance–rejection on $\beta = y - s$:

1. draw β uniform on $[\max(-3, -s), 3]$;
2. accept with probability $p(\beta) = \dfrac{2(\beta + s)}{s + \sqrt{s^2 + 2}} \exp\Big(\tfrac{1}{2} + \tfrac{1}{2}s\big(s - \sqrt{s^2 + 2}\big) - \beta^2\Big)$, the density divided by its maximum;
3. otherwise go back to step 1.

The code keeps this as `inflow.sample_normal_beta`: a fixed number of trials with a mask, because a data-dependent loop cannot be JIT-compiled with static shapes. Production uses an exact inverse CDF instead (`inflow.flux_quantile`): 64 bisection steps on the closed-form CDF of (A.4), with an implicit-function derivative, so that $\partial v_n / \partial(T, u)$ is defined.

Placement. The entrant is given a random first-move fraction $\xi \sim \mathcal{U}[0, \Delta t]$ and tracked from the face for that time, like any other particle. Without it all entrants would start on the face and make a density spike there. The inflow face is assumed clear of the body: an entrant that starts inside it is killed by the post-move inside test and counted as a source shortfall (`inflow_shortfall`), which invalidates the step.

A.5 Move: particle tracking with walls and embedded surfaces

Without surfaces the move is $x \leftarrow x + v\,\Delta t$ followed by the face rules (periodic wrap, specular mirror, outflow delete). With walls or a body, each particle follows SPARTA's event loop (`update.cpp`): a particle with remaining time τ (initially Δt, or the first-move fraction ξ for an entrant) repeats the following.

1. Find the earliest event on the segment $x + tv$, $0 < t \le \tau$, among:
   - the box faces: an exact slab test;
   - the body. For a level set, ϕ is sampled along the segment at spacing $\le h_\phi/2$ to find the first sign change $+ \to -$ (outside to inside). The root is refined by bisection plus one Newton step, $t^h = t - \phi/(v \cdot \nabla\phi)$. Only incoming hits ($v \cdot n < 0$) count. For a line or triangle surface the test is ray–element, excluding the element just hit.
2. If there is no event, fly: $x \leftarrow x + \tau v$, done.
3. Otherwise move to the hit, $x \leftarrow x + t^h v$, $\tau \leftarrow \tau - t^h$, and apply the boundary model with the outward (into-the-flow) normal n:

   $$\text{specular:} \quad v \leftarrow v - 2(v \cdot n)\, n, \tag{A.5}$$

   $$\text{diffuse at } T_w: \quad v \leftarrow \sigma_w\big(r\, n + z_1 t_1 + z_2 t_2\big) + u_w, \qquad \sigma_w = \sqrt{kT_w/m},\ r = \sqrt{-2\ln U},\ z_{1,2} \sim \mathcal{N}(0,1), \tag{A.6}$$

   where (t_1, t_2) is any tangent frame. The normal component is Rayleigh distributed, which is the flux-weighted half-Maxwellian (A.4) at $s = 0$. With accommodation coefficient α the wall is diffuse with probability α and specular otherwise. Outflow faces delete the particle, and periodic faces wrap it.
4. Repeat from step 1 with the remaining τ.

The event loop is batched over all particles as follows (the *lane tracker*, `wall_adjoint`, section 9.6.1 of `PSEUDOCODE.md`).

- Pass 0 runs the search over all N. Typically 0.5 % of particles have an event.
- Those particles are compacted by a prefix sum into a lane of static size $B = \max(256, N/32)$.
- Passes $1, \ldots, 31$ run on the lane only. A concave pocket can make a particle bounce 5–7 times in one step.
- A particle still unresolved after 32 passes is deleted and counted (`surface_stuck`), as SPARTA's `MAXSTUCK` does. Lane overflow is an error.

After the move, a particle found inside the body is an error (`inside_body`). The hit list (element, point, velocities, time) is recorded for the adjoint and for the surface tallies.

A.6 Cut cells and split cells

Cells stay Cartesian and the body is embedded in them.

- **Cut volumes.** A cell crossed by the surface is *cut*. Its flow volume V_c is the fraction of a stratified sub-sample of the cell outside the body; this is what NTC uses in (A.11). Cells wholly inside the body have $V_c = 0$ and never collide.
- **Split cells.** If the body cuts a cell into two or more disconnected flow regions (a notch), each region becomes its own *sub-cell* with its own volume and particle list, as SPARTA does. Particles on opposite sides of a thin wall must not collide with each other. Tracking still uses the Cartesian cell; only the collision cell (`grid.collision_cell`) resolves the region.

A.7 Collisions: No-Time-Counter (NTC) with the VSS model

The VSS molecular model. For a pair with relative speed $g = |v_i - v_j|$ and reduced mass $m_r = m_i m_j/(m_i + m_j)$, the variable-soft-sphere total cross section is

$$\sigma_T(g) = \frac{\pi d_{\mathrm{ref}}^2}{\Gamma(\frac{5}{2} - \omega)} \left(\frac{2kT_{\mathrm{ref}}}{m_r g^2}\right)^{\omega - \frac{1}{2}}, \qquad q(g) \equiv \sigma_T(g)\, g = C_{\mathrm{VSS}}\, (g^2)^{1-\omega} \quad (\texttt{vss.collision_rate}). \tag{A.7}$$

The exponent ω is the viscosity–temperature exponent ($\mu \propto T^{\omega}$; $\omega = \frac{1}{2}$ is hard spheres). Argon (SPARTA `ar.vss`): $d_{\mathrm{ref}} = 4.11 \times 10^{-10}$ m, $\omega = 0.81$, $T_{\mathrm{ref}} = 273.15$ K, $\alpha = 1.40$. The deflection angle χ and azimuth ε of the relative velocity are drawn as

$$\cos\chi = 2U_1^{1/\alpha} - 1, \qquad \varepsilon = 2\pi U_2 \tag{A.8}$$

($\alpha = 1$ is isotropic VHS scattering; $\alpha > 1$ biases forward). With $g = v_i - v_j$ and $g_\perp = \sqrt{g_y^2 + g_z^2}$, the post-collision relative velocity is (Bird eq. 2.22, `vss.scatter`)

$$\begin{aligned} g_x' &= \cos\chi\, g_x + \sin\chi\, \sin\varepsilon\, g_\perp, \\ g_y' &= \cos\chi\, g_y + \sin\chi\, \big(|g|\, g_z \cos\varepsilon - g_x g_y \sin\varepsilon\big)/g_\perp, \\ g_z' &= \cos\chi\, g_z - \sin\chi\, \big(|g|\, g_y \cos\varepsilon + g_x g_z \sin\varepsilon\big)/g_\perp, \end{aligned} \tag{A.9}$$

so $|g'| = |g|$. The centre-of-mass velocity is kept:

$$v_i' = v_{\mathrm{cm}} + \frac{m_j}{m_i + m_j}\, g', \qquad v_j' = v_{\mathrm{cm}} - \frac{m_i}{m_i + m_j}\, g', \qquad v_{\mathrm{cm}} = \frac{m_i v_i + m_j v_j}{m_i + m_j}. \tag{A.10}$$

Momentum and energy are therefore conserved exactly per collision. When $g_\perp \approx 0$, a fallback frame is used.

NTC: how many pairs to test. The expected number of real collisions in cell c over Δt is $\frac{1}{2}n^2\langle\sigma_T g\rangle V_c\Delta t$. With $n = N_c F_{\mathrm{num}}/V_c$ this is $\frac{1}{2}N_c^2 F_{\mathrm{num}}\langle q\rangle\Delta t/V_c$ in simulator pairs. Testing every one of the $\frac{1}{2}N_c(N_c-1)$ pairs would cost $O(N_c^2)$. Bird's no-time-counter scheme instead tests

$$n_{\mathrm{att},c} = \mathrm{round}\left(\tfrac{1}{2}N_c(N_c-1)\,\Sigma_c\,\Delta t\,F_{\mathrm{num}}/V_c\right) \tag{A.11}$$

random pairs. Each is accepted with probability $q(g_{ij})/\Sigma_c$, where $\Sigma_c \geq \max q$ is a majorant of $\sigma_T g$ in the cell. Then $\mathbb{E}[\text{accepted}] = n_{\mathrm{att}}\langle q\rangle/\Sigma_c$ is exactly the collision count above, at $O(N_c)$ cost. This is thinning of a Poisson process with rate bounded by Σ_c: the result is exact whenever $q \leq \Sigma_c$ holds.

- **The majorant.** SPARTA ratchets Σ_c up to the largest q seen. This code uses an a-priori thermal bound $\Sigma_c = 2\,C_{\mathrm{VSS}}\,(25\cdot 2kT_c/m_r)^{1-\omega}$ (a $5v_{\mathrm{th}}$ relative speed, safety factor 2, `ntc.thermal_majorant`), at least the ratcheted value. Violations ($q > \Sigma_c$) are counted and must be zero.
- **Rounding.** The round in (A.11) is stochastic, which keeps it unbiased and gives the count a probability the adjoint can score.

The collision algorithm in one cell (`ntc.collide`, `ntc._slot`):

1. $N_c \leftarrow$ number of particles in the cell; if $N_c < 2$ or $V_c = 0$, stop.
2. $\Sigma_c \leftarrow \max(\text{carried ratchet},\ \text{thermal bound at } T_c)$, held fixed for the step.
3. $n_{\mathrm{att}} \leftarrow \lfloor f\rfloor + \mathbf{1}[U < f - \lfloor f\rfloor]$ with f the argument of (A.11).
4. For $a = 0, 1, \ldots, n_{\mathrm{att}} - 1$, in order:
   (a) *Select a pair* uniformly with replacement across attempts: $i \sim \mathcal{U}\{0..N_c-1\}$, $j = (i+1+r) \bmod N_c$ with $r \sim \mathcal{U}\{0..N_c-2\}$, so $j \neq i$ is uniform on the rest.
   (b) $g \leftarrow |v_i - v_j|$ from the *current* velocities (a particle may collide more than once per step), $q \leftarrow q(g)$ by (A.7).
   (c) *Accept* if $U < q/\Sigma_c$ (and count a violation if $q > \Sigma_c$).
   (d) If accepted, draw (χ, ε) by (A.8) and replace (v_i, v_j) by (A.10).
5. Update the carried ratchet with the largest q tested.

In the code all cells run at once. The loop over a is a `lax.scan` over a static maximum $A_{\max}$ of attempt slots, and a slot is inactive in cells with $a \geq n_{\mathrm{att},c}$. If a cell needs more than $A_{\max}$ attempts, the excess is counted (`attempts_clipped`, must be zero). All random numbers for all slots are drawn in one batch before the scan.

A.8 Sampling: fields and surface loads

Fields are cell averages accumulated over steps, with N_c the particles and V_c the flow volume of cell c:

$$n_c = \frac{F_{\mathrm{num}}\,\langle N_c\rangle}{V_c}, \qquad u_c = \langle v\rangle_c, \qquad T_c = \frac{m}{3k}\left\langle |v-u_c|^2\right\rangle_c. \tag{A.12}$$

Surface loads are momentum and energy tallies of the hits. The force on the body over a window of W steps is

$$\mathbf{F} = \frac{F_{\mathrm{num}}}{W\Delta t}\sum_{k\in\mathrm{window}}\ \sum_{\mathrm{hits}} m\left(v^{\mathrm{in}} - v^{\mathrm{out}}\right), \tag{A.13}$$

and its x-component is the drag objective of the optimiser (section 7.1). Pressure and shear per surface element are the same sum restricted to the element and divided by its area.

A.9 The SPARTA reference cases

Circle:

- 2-D box $[0, 10]^2$ m on a 20×20 grid ($h = 0.5$), body $r = 3$ at $(5, 5)$;
- boundaries: inflow at x_{lo}, outflow at x_{hi}, specular in y;
- argon at $n = 1\,\mathrm{m}^{-3}$, $T = 273.15\,\mathrm{K}$, $u = 100\,\mathrm{m/s}$ ($s \approx 0.30$);
- $F_{\mathrm{num}} = 10^{-3}$, $\Delta t = 10^{-4}\,\mathrm{s}$, 1000 steps from empty; $\approx 43\,000$ particles at steady state;
- almost no collisions, so the case is dominated by the body and the boundaries and V_c matters little.

The sphere is the 3-D analogue: box $[-2, 2]^3$, 20^3 cells, radius 1. The $\mathrm{Kn} \approx 0.5$ circle raises n so that $\lambda \approx 2r$, where the collision step and the cut volumes matter.